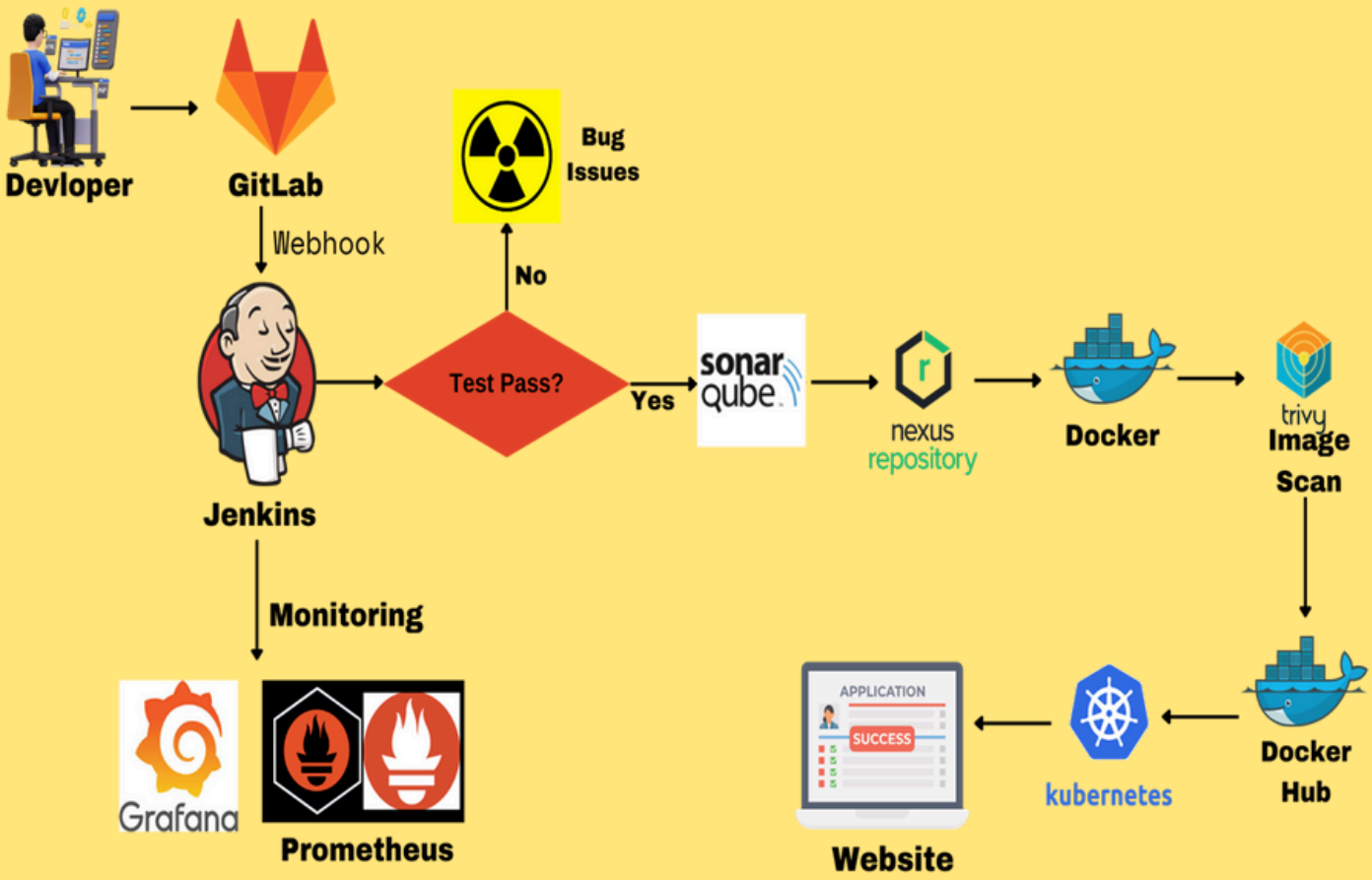
DevOps Shack

Streamlined CI/CD with GitLab

Building Faster, Cleaner, and More Reliable Releases

Introduction

This document provides a detailed description of a CI/CD pipeline that integrates various tools to automate code development, testing, security scanning, deployment, and monitoring. The pipeline is centered around GitLab for version control, Jenkins for continuous integration, SonarQube for code quality analysis, Nexus for artifact storage, Docker for containerization, Trivy for security scanning, and Prometheus/Grafana for monitoring



Here are the tools used in the workflow:

1. IDE/Editor (e.g., VS Code, IntelliJ)
2. Git/GitLab
3. Jenkins
4. SonarQube
5. Nexus Repository
6. Kubernetes/EKS (Elastic Kubernetes Service)
7. Prometheus/Grafana

And here's a brief overview of how GitLab works:

GitLab:

GitLab is a web-based platform for version control and collaboration. It allows developers to manage their codebase, track changes, and collaborate with others. Here's how it works:

1**. Repository**: A repository is created to store the codebase.

2. **Commits**: Developers make changes to the code and commit them to the repository.

3. **Push**: The committed changes are pushed to the GitLab repository.

4. **Pull Request**: Other developers can create pull requests to review and merge changes.

5**. Merge**: Changes are merged into the main branch (e.g., master) after approval.

6**. Issues**: Developers can track issues and bugs in the project using GitLab's issue tracking feature.

7. **Code Review:** Developers can review each other's code and provide feedback.

In this workflow, GitLab is used as the central repository for version control and collaboration. The GitLab repository is connected to Jenkins, which triggers the build process whenever new code is pushed to the repository.

Working:

Step 1: Code Development

- **Tool: IDE/Editor (e.g., VS Code, IntelliJ)**
- **Purpose:** Developers write and edit code in their preferred Integrated Development Environment (IDE) or text editor.
- **Description:** This is the starting point where the actual coding happens. Developers implement new features, fix bugs, and make changes to the codebase.

2. **Step 2: Commit Code**
   - **Tool: Git/GitLab**
   - **Purpose:** Version control and collaboration.
   - **Description:** Once the code is written or modified, it is committed to a local Git repository and then pushed to a remote GitLab repository. This allows multiple developers to collaborate, track changes, and maintain the project's history.
3. **Step 3: Build & Test Code**
   - **Tool: Jenkins**
   - **Purpose:** Continuous Integration (CI) automation.
   - **Description:** Jenkins automatically triggers a build process whenever new code is pushed to the GitLab repository. It compiles the code, runs unit tests, and packages the application. If the build or tests fail, Jenkins notifies the developers to fix the issues.

4. **Step 4: Code Analysis**
    - **Tool: SonarQube**
    - **Purpose:** Ensure code quality.
    - **Description:** SonarQube analyzes the code for quality issues, such as code smells, bugs, vulnerabilities, and duplicate code. It generates a report, which Jenkins can integrate into the build process to enforce quality gates (e.g., the build fails if the code quality does not meet predefined standards).
5. **Step 5: Store Build Artifacts**
    - **Tool: Nexus Repository**
    - **Purpose:** Artifact management.
    - **Description:** After a successful build, Jenkins stores the build artifacts (e.g., .jar, .war files) in Nexus, a repository for binaries and other build artifacts. This central storage allows easy access to specific versions of the application for deployment or rollback if needed.
3. **Step 6: Deploy Application**
    - **Tool: Kubernetes/EKS (Elastic Kubernetes Service)**
    - **Purpose:** Application deployment and orchestration.
    - **Description:** The stored artifacts are deployed to a Kubernetes cluster, which might be managed using Amazon's EKS service. Kubernetes automates the deployment, scaling, and management of containerized applications, ensuring they run reliably across different environments.

Git lab CICD Project

How the Workflow Integrates:

- **Coding:** Developers work in their IDE to create the application.
- **Version Control:** They commit changes to Git, pushing them to GitLab.
- **Automation:** Jenkins automates the build and testing process, integrating code quality checks through SonarQube.
- **Artifact Storage:** Successful builds are stored in Nexus for deployment.
- **Deployment:** Kubernetes handles deploying the application to a cluster, ensuring high availability and scalability.
- **Monitoring:** Prometheus and Grafana continuously monitor the application's health and performance, allowing for proactive management.

Conclusion:

This CI/CD pipeline leverages a suite of powerful tools to automate and streamline the software development lifecycle. By integrating GitLab with Jenkins, SonarQube, Nexus, Docker, Trivy, Prometheus, and Grafana, the pipeline ensures that code is continuously built, tested, analyzed, secured, deployed, and monitored with minimal manual intervention. This not only accelerates the development process but also enhances the quality and security of the application.

For complete CI-CD setup -